

Manual de rol — Persona B

Conducta y Reportes

Cresco · Versión 1.2 · 8 de agosto de 2026 · Envío del equipo: 28 de septiembre

Corrige la v1.0 del 3 de agosto de 2026. Ver `docs/99-archivo/ERRATA-2026-08.md` para el detalle de cada cambio. Esta versión 1.2 reinicia el calendario (tu semana 1 empieza el 9 de agosto) y fija dónde vive `lib/puntaje.ts` : dentro de `servidor/` .

Tu rol en una frase

Tú construyes el corazón funcional del producto.

Todo lo que hace que Cresco sea Cresco y no otra app de mensajería pasa por tu módulo: el puntaje, las acciones, los reportes. Eres quien más lógica de negocio escribe y quien más pruebas necesita, porque un error tuyo no se ve como un error — se ve como un estudiante con el puntaje equivocado, y eso es exactamente lo que un padre va a reclamar.

Ventaja de tu rol: **puedes empezar el día 1 sin depender de nadie.** El motor de puntaje es lógica pura y se prueba sin base de datos ni servidor.

Parte 1 — Contexto del proyecto

1.1 Qué construimos

App Android de comunicación entre docentes y representantes legales de escuelas del Ecuador, para seguimiento de conducta. El estudiante nunca usa la app.

1.2 Decisiones ya tomadas

#	Decisión
ADR-001	PostgreSQL + Drizzle ORM
ADR-002	Next.js como servidor, Expo para las apps
ADR-003	BetterAuth con plugin de organización
ADR-004	Permisos en dos capas: código + RLS
ADR-005	El puntaje es derivado, nunca un contador ← tu ADR
ADR-006	RevenueCat es la fuente de verdad de pagos
ADR-007	Entrega sin tienda, repositorio abierto bajo AGPL-3.0

Parte 2 — Las reglas que debes saber de memoria

2.1 El puntaje

Concepto	Valor
Base al iniciar el parcial	60
Piso	0
Techo	100
Se reinicia	Cada parcial, siempre vuelve a 60

La regla más importante de tu módulo (ADR-005):

```
puntaje = clamp(60 + Σ puntos de acciones VIGENTES del período, 0, 100)
```

El puntaje **nunca se incrementa**: se **recalcula** desde cero cada vez. La tabla `puntaje_periodo` es una caché reconstruible. La verdad son las acciones.

Por qué importa: si fuera un contador, cualquier fallo lo desincronizaría de la bitácora de forma permanente y sin manera de detectarlo. Un padre vería un puntaje que no cuadra con la lista de acciones y tendría razón en reclamar.

2.2 Valores de las acciones

Tipo	Puntos	Quién ajusta
Positiva	+1 por defecto, hasta +2	El docente elige
Negativa · responsabilidad	siempre -1	Fijo, no se ajusta
Negativa · disciplina	-1 por defecto, hasta -3	El docente elige
Nota	0	—

2.3 Topes diarios

- Máximo **+4** por estudiante por día
- Máximo **-5** por estudiante por día

Esto no lo valida la base de datos. Va en tu función de servidor, antes de insertar. Si se excede, devuelves `422` con código `TOPE_DIARIO_ALCANZADO`.

2.4 Los cinco tipos de acción

Tipo	Tabla	Alcance	Bitácora	Reclamable
Positiva	accion_registrada	Un estudiante	Sí	No
Negativa	accion_registrada	Un estudiante	Sí	Sí
Nota	accion_registrada	Un estudiante	No	No
Anuncio	comunicado_curso	Todo el curso	No	No
Evento	comunicado_curso	Todo el curso	No	No

2.5 Estados de una acción

Estado	Puntos que cuenta	Se muestra como
VIGENTE	Los aplicados	Normal
ANULADA	0	"Anulada por el docente"
MODIFICADA	0	"Resuelta con el representante"

Solo **VIGENTE** cuenta en el recálculo. Las otras dos valen 0 pero se muestran distinto al padre. **Solo las negativas se pueden anular.**

2.6 Las seis franjas

Rango	Franja
0 – 15	Situación crítica
16 – 30	Muy por debajo
31 – 50	Por debajo de lo esperado
51 – 60	En el punto de partida

61 – 80	Buen desempeño
81 – 100	Excelente

2.7 Reportes

- El reporte general es **opcional**. Ningún campo es obligatorio.
- Si el docente no lo llena pero hubo acciones, **igual se genera** el reporte del estudiante con solo sus acciones.
- Solo el **más reciente** es visible en la pantalla principal del representante.
- Historial: plan gratuito **2** reportes anteriores, premium **7**.
- El representante **sí ve** los puntos de cada acción.
- Las **notas no entran** en la bitácora del acumulado.

2.8 Eventos

Aparecen como bloque expandible al final del reporte diario, desde hasta 7 días antes de su fecha y hasta el día siguiente. Alcanzan a todo el curso.

Parte 3 — Tu territorio

3.1 Tus tablas

Archivo: `db/schema/conducta.ts`

`categoria_accion` · `tipo_accion` · `accion_registrada` · `comunicado_curso` · `franja_conducta` ·
`puntaje_periodo` · `registro_asistencia` · `plantilla_reporte` · `plantilla_campo` ·
`reporte_general` · `reporte_general_valor` · `reporte_estudiante` · `reporte_estudiante_item` ·
`entrega_reporte`

`pregunta_reporte` existe en tu archivo pero está **diferida a la v2**. No la uses.

Nuevo desde el 8 de agosto: `categoria_accion.aplica_a` queda **reservado sin CHECK** hasta que se retome el rol `DOCENTE_COLABORADOR` en v2 (fue diferido). No generes datos semilla asumiendo valores más allá del default `'AMBAS'`.

3.2 Tus pantallas

#	Pantalla	App	Semana
D11	Asignar acción	Docente	2
D13	Redactar reporte general	Docente	3
P4	Reporte del día (pantalla principal)	Representante	3
P5	Reportes anteriores	Representante	4
P6	Reporte acumulado con gráfico	Representante	4
D12	Tomar asistencia	Docente	5
D14	Publicar anuncio o evento	Docente	5

3.3 Tus endpoints

```
GET  /docente/catalogo-acciones
POST /docente/acciones
POST /docente/acciones/{id}/resolver
POST /docente/cursos/{id}/comunicados
PUT  /docente/cursos/{id}/asistencia
```

```
GET /docente/cursos/{id}/reporte-general
POST /docente/cursos/{id}/reporte-general
GET /representante/reportes-diario
GET /representante/reportes-anteriores
GET /representante/reportes-acumulado
```

3.4 Lo que NO tocas

`db/schema/nucleo.ts` (de A) · `db/schema/interaccion.ts` (de C) · las pantallas de registro, curso, citas, alertas o pagos.

Regla crítica que debes respetar: está prohibido usar `db.select()` suelto sobre `estudiante`, `matricula`, `accion_registrada` o `reporte_estudiante`. Toda consulta pasa por las funciones de `db/acceso/` que construye A. Si no existe la que necesitas, se la pides.

3.5 Dónde vive tu código

`lib/puntaje.ts` vive **dentro de** `servidor/` — el proyecto Next que C scaffoldea el domingo 9 de agosto (día 1 de la semana 1). No es una carpeta que tú creas ni que existe hoy; nace ese día. Tus pruebas de la semana 1 (martes y miércoles, "La función de cálculo") son lógica pura y no necesitan que `servidor/` exista todavía para escribirse y correr con `node --test`, pero el archivo final se guarda ahí una vez que la carpeta nace.

No hace falta compartir este archivo con `movil/`: el servidor calcula el puntaje y lo devuelve ya resuelto en `puntaje_actual` dentro de la respuesta de `POST /docente/acciones`. El móvil nunca recalcula, solo muestra el número que llega.

Parte 4 — Preparación (semana 1, 4 de agosto en adelante)

4.1 Instalar

Node.js 20+, Git, VS Code con ESLint y Prettier.

```
node --version    # v20 o superior
git config --global user.name "Tu Nombre"
git config --global user.email "tu@correo.com"
```

4.2 Conceptos que necesitas

¿Qué es una prueba unitaria? Código que verifica que otro código hace lo que debe. Tú vas a escribir muchas, porque tu módulo tiene reglas numéricas exactas que un error silencioso puede romper sin que nadie lo note.

¿Qué es una función pura? Una que solo depende de sus argumentos y no toca base de datos ni red. El cálculo del puntaje debe ser una función pura: recibe una lista de acciones y devuelve un número. Así la pruebas sin levantar nada.

¿Qué es el servidor simulado? Mientras A construye la base de datos real, tú puedes trabajar tus pantallas contra datos falsos:

```
npx @stoplight/prism-cli mock api/openapi.yaml
```

4.3 Clonar

```
git clone <url-del-repo>
cd cresco
npm ci                    # versiones exactas del lockfile
```

Parte 5 — Semana a semana

Semana 1 · 9 al 15 de agosto — El motor

No necesitas esperar a nadie esta semana. Todo lo tuyo es lógica pura.

Domingo 9 y lunes 10 — La función de cálculo

Crea `lib/puntaje.ts` (va dentro de `servidor/` en cuanto C lo scaffoldee el domingo — ver §3.5) con una función pura:

```
type AccionParaCalculo = {
  puntosAplicados: number;
  estado: "VIGENTE" | "ANULADA" | "MODIFICADA";
};

export function calcularPuntaje(
  acciones: AccionParaCalculo[],
  base = 60,
  minimo = 0,
  maximo = 100,
): number {
  const suma = acciones
    .filter((a) => a.estado === "VIGENTE")
    .reduce((total, a) => total + a.puntosAplicados, 0);
  return Math.min(maximo, Math.max(minimo, base + suma));
}
```

Martes 11 — Los topes diarios

Crea la función que decide si una acción se puede registrar:

```
export function puedeRegistrar(
  accionesDelDia: AccionParaCalculo[],
  puntosNuevos: number,
  topePositivo = 4,
  topeNegativo = 5,
): { permitido: boolean; motivo?: string } {
  // suma solo VIGENTE, separa positivos de negativos,
  // verifica que sumar puntosNuevos no exceda el tope correspondiente
}
```

Miércoles 12 — Pruebas

Escribe al menos estas ocho:

#	Caso	Esperado
1	Sin acciones	60
2	Una positiva de +2	62
3	Una negativa de -3	57
4	Acciones que suman -70	0, no negativo
5	Acciones que suman +50	100, no 110
6	Una acción ANULADA	No cuenta
7	Ya perdió -5 hoy, intenta otra -1	Rechazado
8	Ya ganó +4 hoy, intenta otra +1	Rechazado

Éxito de la semana 1: `npm test` pasa las ocho.

Jueves 13 y viernes 14 — Catálogo semilla

Coordina con A, que está armando `db/seeds/`. Tú defines los tipos de acción con sus puntos exactos. Como mínimo:

- **Positivas:** excelente taller, excelente lección, excelente examen, participación destacada, ayudó a un compañero
- **Responsabilidad (-1 fijo):** no trajo la tarea, no trajo materiales, tarea incompleta
- **Disciplina (-1 a -3):** interrumpió la clase, no acató indicaciones, falta de respeto a un compañero, falta de respeto al docente
- **Notas (o):** observación general

Semana 2 · 16 al 22 de agosto — Asignar acción

Pantalla D11

El flujo del docente, pensado para hacerse en segundos entre clase y clase:

1. Elige el tipo de acción
2. Busca al estudiante (barra de búsqueda, la lista puede tener 40 nombres)
3. Ajusta los puntos si el tipo lo permite

4. Confirma

Trampa: valida en el servidor, no solo en la pantalla. Un cliente modificado no debe poder registrar -10 puntos.

Endpoint `POST /docente/acciones`

Debe verificar, en este orden:

1. ¿El docente tiene asignación vigente en ese curso? Si no → `403`
2. ¿El período está abierto y es día de clase? Si no → `409`
3. ¿Los puntos están dentro del rango del tipo? Si no → `422`
4. ¿Se excede el tope diario? Si sí → `422 TOPE_DIARIO_ALCANZADO`
5. Insertar y recalcular

Éxito de la semana 2: asignar -1 a un estudiante actualiza su puntaje de 60 a 59 en pantalla, y un segundo intento que exceda -5 en el día es rechazado con un mensaje claro.

Semana 3 · 23 al 29 de agosto — Reportes

Semana del hito crítico del proyecto.

Pantalla D13 — Reporte general

Campos etiquetados que vienen de `plantilla_campo`. Propuesta inicial de campos:

Código	Etiqueta
ANUNCIOS	Anuncios
NOVEDADES	Novedades del día
TAREAS	Tareas enviadas
CONSEJO	Consejo para casa

Ninguno es obligatorio. El docente debe poder publicar con un solo campo lleno, o guardarlo como borrador.

Importante: los campos vienen de la tabla, no están escritos en el código. Agregar "material para mañana" debe ser un `INSERT`, no un cambio de código.

Pantalla P4 — Reporte del día (la más importante del proyecto)

Es lo primero que ve el representante al abrir la app. Debe leerse en menos de un minuto.

Situación	Qué muestra
Hay reporte de hoy	El reporte completo
Aún no se genera (204)	"Aún no tienes reporte nuevo, pero puedes ver el anterior" + botón
Sin novedades	Solo el contenido general del curso
Con acciones	Las acciones con sus puntos + puntaje nuevo + franja
Hay evento vigente	Bloque expandible al final

Consejo: el padre ecuatoriano promedio no lee gráficos estadísticos. Un número grande, una frase clara y colores que se entiendan sin leer. Nada de percentiles ni desviaciones.

Éxito de la semana 3: el docente publica el reporte desde su teléfono y el representante lo ve en el suyo.

Semana 4 · 30 de agosto al 5 de septiembre — Historial y acumulado

Pantalla P5 — Reportes anteriores

Límite según plan: 2 en gratuito, 7 en premium. Al llegar al límite, muestra la invitación a premium. Coordina con C, que construye el paywall.

Pantalla P6 — Reporte acumulado

- Puntaje actual grande, con su franja y frase
- Gráfico simple de evolución (una línea, sin ejes complicados)
- Bitácora: todas las acciones del parcial excepto las notas
- Resumen por categoría: cuántas de responsabilidad, cuántas de disciplina
- Inasistencias y atrasos

Consejo sobre el gráfico: usa una librería simple de React Native. No intentes hacerlo desde cero con SVG. Que se entienda de un vistazo: una línea que sube es bueno, una que baja es malo. Nada más.

Éxito de la semana 4: el gráfico se dibuja con datos reales de dos semanas.

Semana 5 · 6 al 12 de septiembre — Asistencia y comunicados

D12 — Asistencia

Una pantalla, toda la lista, un toque por estudiante. Por defecto todos presentes; el docente solo marca las excepciones.

D14 — Anuncios y eventos

El evento necesita: título, contenido, fecha y hora. La ventana de visibilidad se calcula sola (7 días antes, hasta el día siguiente).

Éxito: un evento aparece y desaparece en las fechas correctas.

Semana 6 · 13 al 19 de septiembre — Cierre

Trabajo conjunto: datos de demostración, corrección de errores, video.

Tú aportas los datos que hacen creíble la demostración: un curso con dos semanas de historial, puntajes variados que muestren al menos cuatro franjas distintas, y una acción anulada para mostrar cómo se ve.

Parte 6 — Trampas y consejos

Nunca incrementes el puntaje. Recalcúlalo siempre. Si te descubres escribiendo `puntaje = puntaje - 1`, estás cometiendo el error que el ADR-005 existe para evitar.

Los puntos aplicados son una fotografía. `accion_registrada.puntosAplicados` guarda el valor del momento del registro. Si el catálogo cambia después, las acciones viejas conservan sus puntos.

Recalcula en tres momentos: al registrar una acción, al resolverla (anular o modificar), y al cerrar el período.

Las notas no van a la bitácora. Aparecen en el reporte del día pero no en el acumulado. Es fácil olvidarlo y llenar la bitácora de ruido.

El reporte del estudiante es inmutable. Una vez generado, es una fotografía del día. No lo regeneres si cambian las acciones después.

Prueba con el caso vacío. Un estudiante sin ninguna acción debe ver un reporte válido, no una pantalla rota. Es el caso más común: la mayoría de los días la mayoría de los niños no tienen novedades.

No construyas nada para `DOCENTE_COLABORADOR`. Está diferido a v2 — incluido cualquier valor de `categoria_accion.aplica_a` más allá de `'AMBAS'`.

Parte 7 — Cuando te bloqueeas

1. **Relee el contrato** en `api/openapi.yaml` .
2. **Revisa las reglas** en la parte 2 de este manual.
3. **Medio día máximo.** Si llevas más, avisa al grupo ese mismo día.
4. **Pregunta a una IA** pegando el bloque de la siguiente sección.

Parte 8 — Bloque de contexto para IA

Copia todo esto y pégalo antes de tu pregunta.

CONTEXTO – PROYECTO CRESCO, ROL: PERSONA B (CONDUCTA Y REPORTES)

Soy uno de 3 estudiantes (equipo Neofix) construyendo una app Android para el hackathon RevenueCat Shipaton 2026, categoría Next Gen. Semana 1 desde el 9 de agosto de 2026; deadline interno 20 de septiembre; envío real 28 de septiembre. Es mi primera app. Mi rol es el módulo de acciones, puntaje, asistencia y reportes.

MI CÓDIGO VIVE EN servidor/lib/puntaje.ts – dentro del proyecto Next que C scaffoldea el día 1. No hace falta compartirlo con movil/: el servidor devuelve el puntaje ya calculado en la respuesta; el móvil solo lo muestra.

EL PRODUCTO: app de comunicación entre docentes y representantes legales de escuelas del Ecuador, para seguimiento de conducta. El estudiante NUNCA es usuario. La interfaz del representante debe ser MUY simple: muchos padres ecuatorianos no leen gráficos estadísticos.

STACK: PostgreSQL + Drizzle ORM. Next.js como servidor. Expo/React Native para las apps. NativeWind para estilos (shadcn NO funciona en React Native).

MIS TABLAS: categoria_accion, tipo_accion, accion_registrada, comunicado_curso, franja_conducta, puntaje_periodo, registro_asistencia, plantilla_reporte, plantilla_campo, reporte_general, reporte_general_valor, reporte_estudiante, reporte_estudiante_item, entrega_reporte.

REGLAS DEL PUNTAJE (las más importantes de mi rol):

- Base 60 por parcial, piso 0, techo 100. Se reinicia cada parcial.
- Positivas: +1 por defecto, ajustable a +2.
- Negativas de responsabilidad: FIJO -1. De disciplina: -1 a -3.
- Topes diarios: máximo +4 y máximo -5 por estudiante por día. Se validan en el servidor, NO en la base de datos. Si se excede: 422 TOPE_DIARIO_ALCANZADO.
- EL PUNTAJE ES DERIVADO, NUNCA UN CONTADOR:
puntaje = clamp(60 + suma de acciones VIGENTES del período, 0, 100)
La tabla puntaje_periodo es una CACHE recalculable. La verdad son las

acciones.

- `accion_registrada.puntos_aplicados` es una fotografía inmutable: cambiar el catálogo no altera acciones ya registradas.
- Estados: VIGENTE (cuenta), ANULADA (0 puntos), MODIFICADA (0 puntos, resuelta con el representante). Solo las NEGATIVAS se pueden anular.
- Se recalcula al registrar, al resolver y al cerrar el período.
- Al cerrar el parcial el puntaje se congela.

5 TIPOS DE ACCIÓN:

- Positiva, Negativa, Nota → tabla `accion_registrada` (por estudiante)
- Anuncio, Evento → tabla `comunicado_curso` (por curso completo)
- Las NOTAS no entran en la bitácora del acumulado.
- Los EVENTOS aparecen en el reporte diario desde 7 días antes hasta el día siguiente, como bloque expandible.

6 FRANJAS: 0-15 crítica, 16-30 muy por debajo, 31-50 por debajo, 51-60 punto de partida, 61-80 buen desempeño, 81-100 excelente.

REPORTES:

- El reporte general del curso es OPCIONAL y ningún campo es obligatorio. Los campos vienen de la tabla `plantilla_campo`.
- Si no hay reporte general pero hubo acciones, igual se genera el del estudiante.
- Si no hay reporte de hoy, el endpoint devuelve 204 y la app muestra "Aún no tienes reporte nuevo, pero puedes ver el anterior".
- Solo el más reciente es visible en pantalla principal. Historial: 2 reportes en plan gratuito, 7 en premium.
- El representante SÍ ve los puntos de cada acción.
- `reporte_estudiante` es una fotografía inmutable del día.

DIFERIDO A V2: el rol `DOCENTE_COLABORADOR` no está activo en v1. `categoria_accion.aplica_a` queda reservado sin CHECK hasta entonces – no generar semillas asumiendo valores más allá de 'AMBAS'.

SEGURIDAD: prohibido usar `db.select()` suelto sobre estudiante, matricula, `accion_registrada` o `reporte_estudiante`. Toda consulta pasa por funciones de `db/acceso/` que garantizan el filtro por vínculo activo.

CONVENCIONES: tablas y columnas en español snake_case. Estados en MAYUSCULA_SNAKE en `db/schema/enums.ts`. Timestamps UTC, zona America/Guayaquil.

MI RESTRICCIÓN: 6 semanas, primera app, prefiero soluciones simples y probadas. Explícame los conceptos, no solo el código.

